



K. R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

IMAGE PROCESSING & COMPUTER VISION LAB FILE

(ENSP354)

Lab file Submitted to

K. R. Mangalam University

for

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Shreyash Jha (2301010110) (Sem-06)

Course Teacher

Feroz Ahmad

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

Sohna, Haryana 122103, India

Index

NO.	EXPERIMENTS
1.	Implement a program to acquire and display an image. Demonstrate the process of image sensing and acquisition, and explain the components involved in an image processing system
2.	Develop a program to perform sampling and quantization on a given image. Visualize the effects of different sampling rates and quantization levels on image quality
3.	Implement image enhancement techniques in the spatial domain, including contrast stretching and histogram equalization. Compare the results before and after enhancement.
4.	Apply frequency domain filtering to an image. Implement both low pass and high pass filters, and demonstrate their effects.
5.	Implement a noise model and apply spatial filtering techniques for image restoration.
6.	Periodic noise reduction using inverse filtering and Wiener filtering.
7.	Implement geometric transformations such as rotation, scaling, and translation.
8.	Perform color image processing by converting an image from RGB to another color model (HSV, YCbCr). Implement and visualize color-based image segmentation techniques.
9.	Implement Huffman Coding and Run Length Coding for image compression. Compare compression ratios.
10.	Perform image segmentation using edge detection techniques.
11.	Apply morphological operations such as dilation and erosion on a binary image.
12.	Implement threshold-based and region-based segmentation techniques.
13.	Implement boundary and regional descriptors using chain codes.
14.	Develop a basic Convolutional Neural Network (CNN) for image classification.
15.	Lossless and Lossy Image Compression Using Huffman Coding
16.	Shannon–Fano Coding and Arithmetic Coding

EXPERIMENT-01

- 1. Implement a program to acquire and display an image. Demonstrate the process of image sensing and acquisition, and explain the components involved in an image processing system.**

CODE:-

```
import cv2  
image = cv2.imread(r'C:/Users/mishr/Downloads/13676.jpg')  
if image is None:  
    print("Error: Image not found")  
else:  
    cv2.imshow('Acquired Image', image)  
    cv2.waitKey(0)  
    cv2.destroyAllWindows()
```



EXPERIMENT-02

2. Develop a program to perform sampling and quantization on a given image. Visualize the effects of different sampling rates and quantization levels on image quality

CODE:-

```
image = cv2.imread('13676.jpg', cv2.IMREAD_GRAYSCALE)
```

```
# Function for sampling
```

```
def sampling(img, factor):  
    return img[::factor, ::factor]
```

```
# Function for quantization
```

```
def quantization(img, levels):  
    img_norm = img / 255.0  
    img_quant = np.floor(img_norm * levels) / levels  
    return (img_quant * 255).astype(np.uint8)
```

```
# Sampling factors and quantization levels
```

```
sampling_factors = [1, 2, 4]  
quant_levels = [256, 64, 16]
```

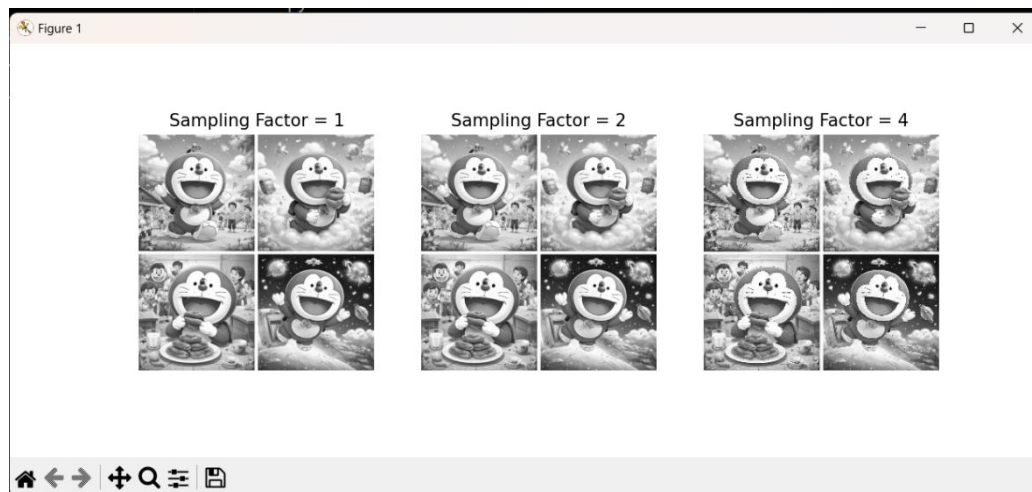
```
# Display sampling results
```

```
plt.figure(figsize=(10, 4))  
for i, f in enumerate(sampling_factors):  
    sampled_img = sampling(image, f)  
    plt.subplot(1, 3, i+1)  
    plt.imshow(sampled_img, cmap='gray')  
    plt.title(f'Sampling Factor = {f}')  
    plt.axis('off')  
plt.show()
```

```
# Display quantization results
```

```
plt.figure(figsize=(10, 4))  
for i, q in enumerate(quant_levels):  
    quant_img = quantization(image, q)  
    plt.subplot(1, 3, i+1)
```

```
plt.imshow(quant_img, cmap='gray')  
plt.title(f'Quantization Levels = {q}')  
plt.axis('off')  
plt.show()
```



EXPERIMENT-03

3. Implement image enhancement techniques in the spatial domain, including contrast stretching and histogram equalization. Compare the results before and after enhancement.

CODE:-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

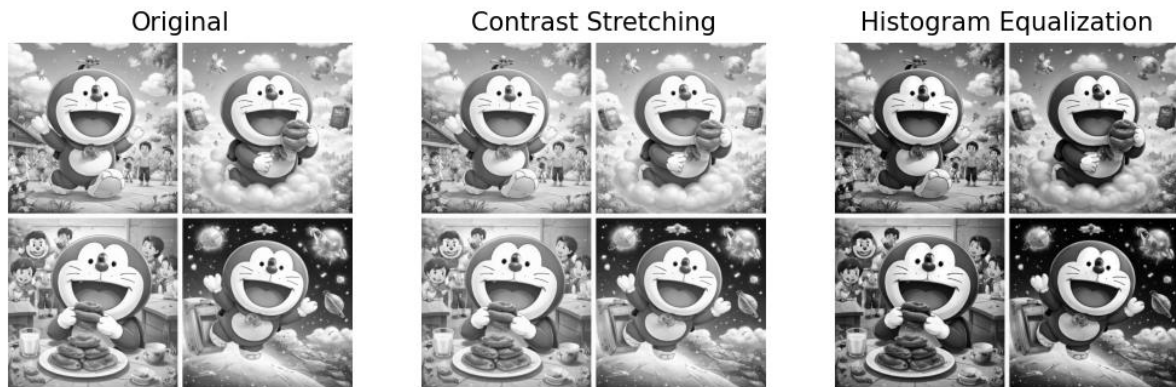
img = cv2.imread('input_image.jpg', 0)

# Contrast Stretching
min_val, max_val = np.min(img), np.max(img)
cs_img = ((img - min_val) / (max_val - min_val) * 255).astype(np.uint8)

# Histogram Equalization
he_img = cv2.equalizeHist(img)

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(img, cmap='gray'); plt.title('Original');
plt.axis('off')
plt.subplot(1,3,2); plt.imshow(cs_img, cmap='gray'); plt.title('Contrast
Stretching'); plt.axis('off')
```

```
plt.subplot(1,3,3); plt.imshow(he_img, cmap='gray'); plt.title('Histogram  
Equalization'); plt.axis('off')  
  
plt.show()
```



EXPERIMENT-04

**4. Apply frequency domain filtering to an image.
Implement both low pass and high pass filters, and
demonstrate their effects.**

CODE:-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('input_image.jpg', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
rows, cols = img.shape
crow, ccol = rows//2, cols//2

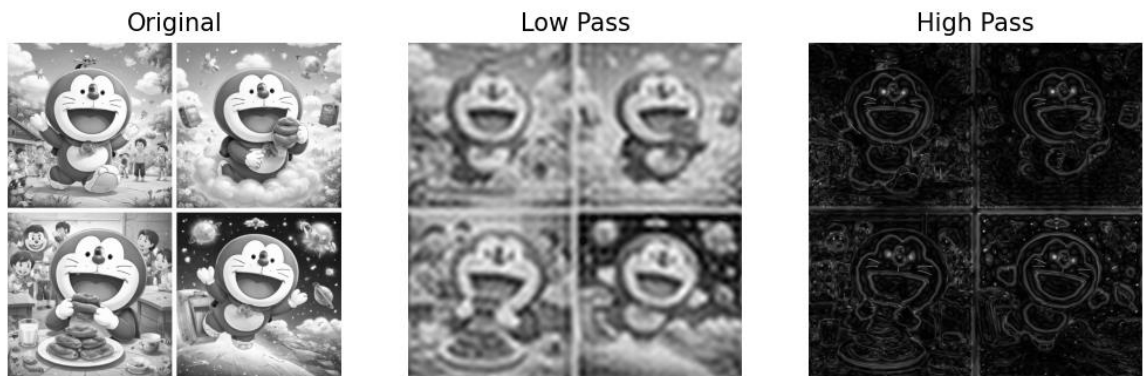
mask_lp = np.zeros((rows, cols), np.uint8)
mask_lp[crow-30:crow+30, ccol-30:ccol+30] = 1
mask_hp = 1 - mask_lp

lp = np.fft.ifft2(np.fft.ifftshift(fshift * mask_lp))
hp = np.fft.ifft2(np.fft.ifftshift(fshift * mask_hp))

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(img, cmap='gray'); plt.title('Original');
plt.axis('off')
```



```
plt.subplot(1,3,2); plt.imshow(np.abs(lp), cmap='gray'); plt.title('Low Pass');  
plt.axis('off')  
  
plt.subplot(1,3,3); plt.imshow(np.abs(hp), cmap='gray'); plt.title('High Pass');  
plt.axis('off')  
  
plt.show()
```



EXPERIMENT-05

5. Implement a noise model and apply spatial filtering techniques for image restoration.

CODE:-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('input_image.jpg', 0)

# Gaussian Noise
gauss = img + np.random.normal(0, 20, img.shape)
gauss = np.clip(gauss, 0, 255).astype(np.uint8)

# Salt & Pepper Noise
sp = img.copy()
prob = 0.02
rand = np.random.rand(*img.shape)
sp[rand < prob] = 0
sp[rand > 1-prob] = 255

mean_filt = cv2.blur(gauss, (3,3))
median_filt = cv2.medianBlur(sp, 3)

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(gauss, cmap='gray'); plt.title('Gaussian Noise');
plt.axis('off')
plt.subplot(1,3,2); plt.imshow(mean_filt, cmap='gray'); plt.title('Mean Filter');
plt.axis('off')
plt.subplot(1,3,3); plt.imshow(median_filt, cmap='gray'); plt.title('Median
Filter'); plt.axis('off')
plt.show()
```

Gaussian Noise



Mean Filter



Median Filter



EXPERIMENT-06

6. Periodic noise reduction using inverse filtering and Wiener filtering.

CODE:-

```
from scipy.signal import wiener
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('input_image.jpg', 0)
noise = 20 * np.sin(np.linspace(0,50,img.shape[1]))
noise = noise.reshape(1,-1)
noisy = img + noise
noisy = np.clip(noisy,0,255).astype(np.uint8)

wiener_img = wiener(noisy)

plt.imshow(wiener_img, cmap='gray')
plt.title('Wiener Filtered Image')
plt.axis('off')
plt.show()
```



EXPERIMENT-07

7. Implement geometric transformations such as rotation, scaling, and translation.

CODE:-

```
import cv2
import numpy as np

img = cv2.imread('input_image.jpg')
rows, cols = img.shape[:2]

M_trans = np.float32([[1,0,50],[0,1,50]])
translated = cv2.warpAffine(img, M_trans, (cols, rows))

scaled = cv2.resize(img, None, fx=0.5, fy=0.5)

M_rot = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated = cv2.warpAffine(img, M_rot, (cols, rows))

cv2.imshow('Translated', translated)
cv2.imshow('Scaled', scaled)
cv2.imshow('Rotated', rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

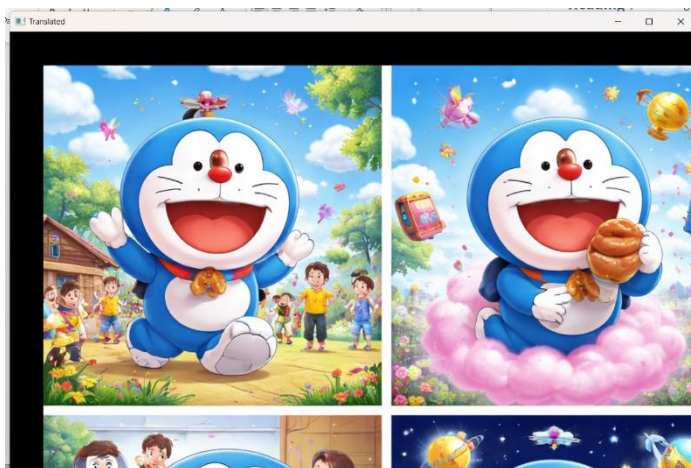
ROTATED:-



SCALED:-



TRANSLATED:-



EXPERIMENT-08

8. Perform color image processing by converting an image from RGB to another color model (HSV, YCbCr). Implement and visualize color-based image segmentation techniques.

CODE:-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread('input_image.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower = np.array([0, 120, 70])
upper = np.array([10, 255, 255])
mask = cv2.inRange(hsv, lower, upper)
segmented = cv2.bitwise_and(img, img, mask=mask)
plt.subplot(1,2,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));
plt.title('Original'); plt.axis('off')
plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(segmented,
cv2.COLOR_BGR2RGB)); plt.title('Segmented'); plt.axis('off')
plt.show()
```



EXPERIMENT-09

9. Implement Huffman Coding and Run Length Coding for image compression. Compare compression ratios.

CODE:-

```
import cv2
import numpy as np
from collections import Counter
import heapq

img = cv2.imread('input_image.jpg', 0)
flat = img.flatten()

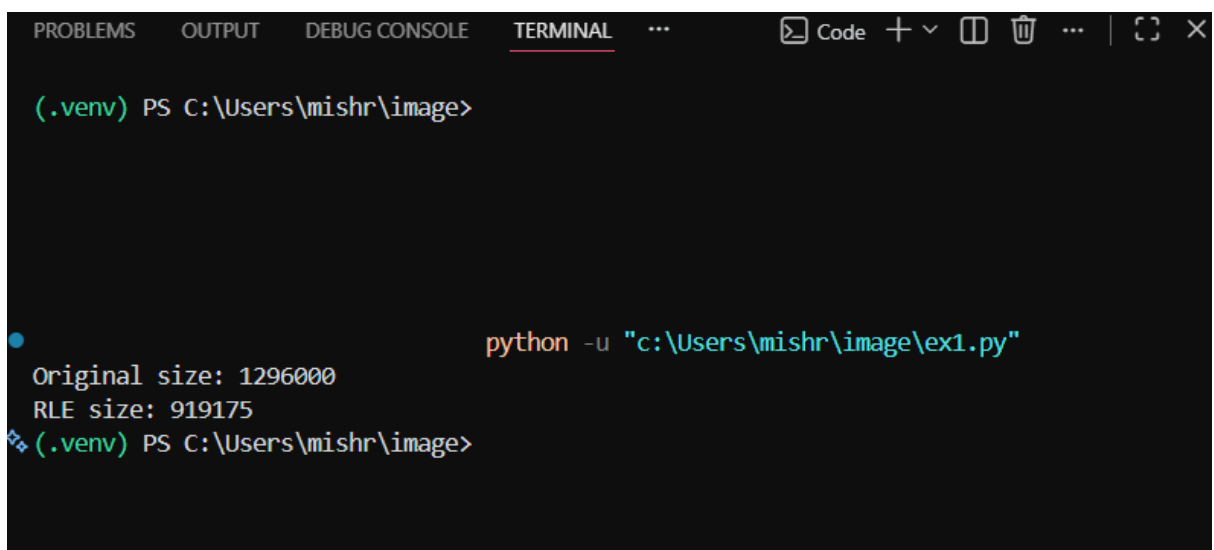
# Run Length Encoding
rle = []
prev = flat[0]
count = 1
for pixel in flat[1:]:
    if pixel == prev:
        count += 1
    else:
        rle.append((prev, count))
        prev = pixel
        count = 1
```



```
rle.append((prev, count))
```

```
print("Original size:", len(flat))
```

```
print("RLE size:", len(rle))
```



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  ...  Code  +  -  [ ]  [X]  ...  |  [ ]  [X]
```

```
(.venv) PS C:\Users\mishr\image>
```

```
python -u "c:\Users\mishr\image\ex1.py"
```

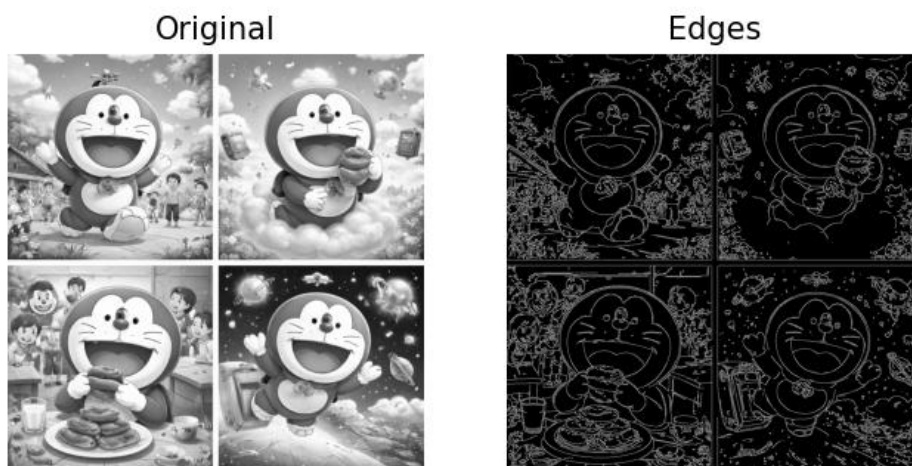
```
Original size: 1296000  
RLE size: 919175  
(.venv) PS C:\Users\mishr\image>
```

EXPERIMENT-10

10. Perform image segmentation using edge detection techniques.

CODE:-

```
import cv2  
  
import matplotlib.pyplot as plt  
  
img = cv2.imread('input_image.jpg', 0)  
edges = cv2.Canny(img, 100, 200)  
  
plt.subplot(1,2,1); plt.imshow(img, cmap='gray'); plt.title('Original');  
plt.axis('off')  
  
plt.subplot(1,2,2); plt.imshow(edges, cmap='gray'); plt.title('Edges');  
plt.axis('off')  
  
plt.show()
```



EXPERIMENT-11

11. Apply morphological operations such as dilation and erosion on a binary image.

CODE:-

```
import cv2
import numpy as np

img = cv2.imread('input_image.jpg', 0)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

kernel = np.ones((5,5), np.uint8)
erosion = cv2.erode(binary, kernel, iterations=1)
dilation = cv2.dilate(binary, kernel, iterations=1)

cv2.imshow('Binary', binary)
cv2.imshow('Erosion', erosion)
cv2.imshow('Dilation', dilation)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

BINARY:-



EROSION:-



DILATION:-



EXPERIMENT:12

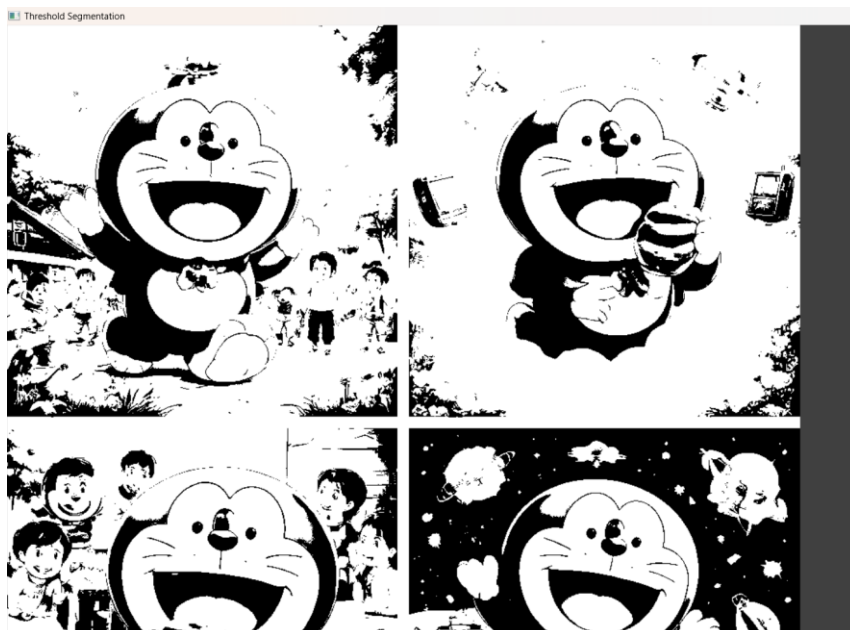
12. Implement threshold-based and region-based segmentation techniques.

CODE:-

```
import cv2
import numpy as np

img = cv2.imread('input_image.jpg', 0)
_, thresh = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

cv2.imshow('Threshold Segmentation', thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



EXPERIMENT-13

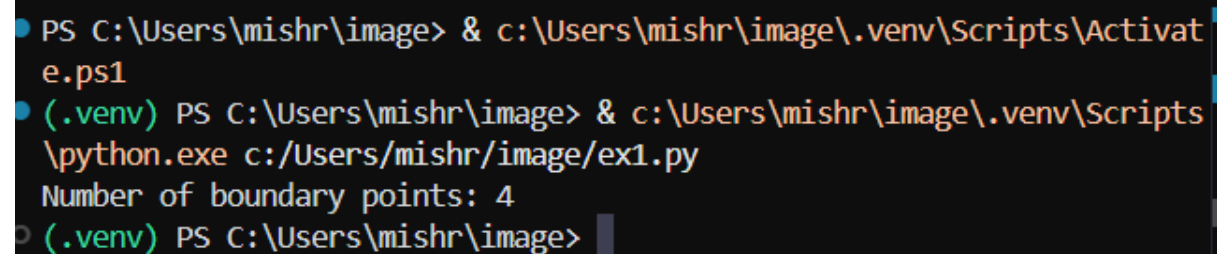
13. Implement boundary and regional descriptors using chain codes.

CODE:-

```
import cv2

img = cv2.imread('input_image.jpg', 0)
_, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_NONE)

print("Number of boundary points:", len(contours[0]))
```



```
• PS C:\Users\mishr\image> & c:\Users\mishr\image\.venv\Scripts\Activate.ps1
• (.venv) PS C:\Users\mishr\image> & c:\Users\mishr\image\.venv\Scripts\python.exe c:/Users/mishr/image/ex1.py
Number of boundary points: 4
• (.venv) PS C:\Users\mishr\image>
```

EXPERIMENT-14

14. Develop a basic Convolutional Neural Network (CNN) for image classification.

CODE:-

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.summary()
```

python -u "c:\Users\mishr\image\ex1.py"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
flatten (Flatten)	(None, 30752)	0
dense (Dense)	(None, 128)	3,936,384
dense_1 (Dense)	(None, 10)	1,290

Total params: 3,938,570 (15.02 MB)
Trainable params: 3,938,570 (15.02 MB)
Non-trainable params: 0 (0.00 B)

PS C:\Users\mishr\image>

EXPERIMENT-15

15. Lossless and Lossy Image Compression Using Huffman Coding

CODE:-

```
import cv2

from collections import Counter

img = cv2.imread('input_image.jpg', 0)

freq = Counter(img.flatten())

print("Pixel Frequencies:", freq)
```

```
PS C:\Users\mishr\image> python -u "c:\Users\mishr\image\ex1.py"
Pixel Frequencies: Counter({np.uint8(8): 66355, np.uint8(7): 64187, np.uint8(9): 63505, np.uint8(10): 61756, np.uint8(6): 60379, np.uint8(5): 57588, np.uint8(4): 52858, np.uint8(11): 50165, np.uint8(3): 47041, np.uint8(2): 43889, np.uint8(12): 43476, np.uint8(1): 40445, np.uint8(13): 38927, np.uint8(14): 35298, np.uint8(0): 34108, np.uint8(15): 32727, np.uint8(16): 30358, np.uint8(17): 28385, np.uint8(18): 26906, np.uint8(19): 25624, np.uint8(20): 24282, np.uint8(21): 22777, np.uint8(22): 21790, np.uint8(23): 20458, np.uint8(24): 19212, np.uint8(25): 1846, np.uint8(26): 17582, np.uint8(27): 16729, np.uint8(28): 15339, np.uint8(29): 14298, np.uint8(30): 13474, np.uint8(31): 12618, np.uint8(32): 11814, np.uint8(33): 11167, np.uint8(34): 10332, np.uint8(35): 9334, np.uint8(36): 8135, np.uint8(37): 7415, np.uint8(38): 6488, np.uint8(39): 5847, np.uint8(40): 5138, np.uint8(41): 4477, np.uint8(42): 3886, np.uint8(43): 3340, np.uint8(44): 3057, np.uint8(45): 2762, np.uint8(46): 2545, np.uint8(47): 2380, np.uint8(48): 2265, np.uint8(49): 1936, np.uint8(50): 1905, np.uint8(51): 1789, np.uint8(52): 1630, np.uint8(53): 1411, np.uint8(54): 1312, np.uint8(55): 1220, np.uint8(56): 1211, np.uint8(57): 1025, np.uint8(59): 920, np.uint8(58): 899, np.uint8(60): 780, np.uint8(230): 749, np.uint8(232): 745, np.uint8(60): 738, np.uint8(231): 699, np.uint8(229): 688, np.uint8(234): 686, np.uint8(61): 683, np.uint8(237): 677, np.uint8(236): 675, np.uint8(235): 663, np.uint8(227): 660, np.uint8(240): 652, np.uint8(228): 650, np.uint8(233): 648, np.uint8(63): 641, np.uint8(64): 628, np.uint8(239): 627, np.uint8(233): 623, np.uint8(66): 579, np.uint8(225): 578, np.uint8(226): 577, np.uint8(241): 574, np.uint8(65): 567, np.uint8(67): 563, np.uint8(224): 562, np.uint8(243): 540, np.uint8(242): 539, np.uint8(69): 527, np.uint8(223): 509, np.uint8(222): 504, np.uint8(68): 500, np.uint8(244): 495, np.uint8(245): 493, np.uint8(71): 492, np.uint8(70): 484, np.uint8(73): 472, np.uint8(72): 471, np.uint8(246): 467, np.uint8(221): 460, np.uint8(74): 456, np.uint8(220): 443, np.uint8(218): 435, np.uint8(75): 433, np.uint8(76): 424, np.uint8(219): 416, np.uint8(247): 411, np.uint8(78): 409, np.uint8(79): 400, np.uint8(216): 393, np.uint8(217): 390, np.uint8(77): 385, np.uint8(81): 370, np.uint8(82): 370, np.uint8(255): 367, np.uint8(214): 362, np.uint8(83): 361, np.uint8(80): 355, np.uint8(213): 351, np.uint8(85): 348, np.uint8(86): 348, np.uint8(90): 346, np.uint8(88): 336, np.uint8(215): 333, np.uint8(84): 333, np.uint8(248): 329, np.uint8(210): 325, np.uint8(212): 322, np.uint8(91): 315, np.uint8(92): 307, np.uint8(249): 307, np.uint8(87): 306, np.uint8(89): 306, np.uint8(207): 298, np.uint8(94): 287, np.uint8(211): 285, np.uint8(97): 281, np.uint8(93): 278, np.uint8(209): 274, np.uint8(95): 271, np.uint8(99): 269, np.uint8(250): 269, np.uint8(96): 262, np.uint8(98): 262, np.uint8(100): 262, np.uint8(206): 262, np.uint8(202): 261, np.uint8(204): 246, np.uint8(203): 243, np.uint8(205): 242, np.uint8(208): 242, np.uint8(200): 241, np.uint8(201): 240, np.uint8(104): 234, np.uint8(108): 234, np.uint8(101): 233, np.uint8(109): 229, np.uint8(199): 223, np.uint8(198): 221, np.uint8(103): 220, np.uint8(251): 218, np.uint8(106): 217, np.uint8(105): 215, np.uint8(112): 215, np.uint8(182): 215, np.uint8(194): 214, np.uint8(110): 213, np.uint8(116): 212, np.uint8(102): 211, np.uint8(196): 211, np.uint8(114): 203, np.uint8(107): 202, np.uint8(192): 202, np.uint8(189): 202, np.uint8(111): 201, np.uint8(193): 200, np.uint8(197): 198, np.uint8(190): 197, np.uint8(117): 194, np.uint8(191): 194, np.uint8(188): 194, np.uint8(113): 193, np.uint8(185): 191, np.uint8(121): 190, np.uint8(187): 190, np.uint8(122): 188, np.uint8(115): 186, np.uint8(120): 185, np.uint8(195): 185, np.uint8(186): 185, np.uint8(172): 183, np.uint8(125): 182, np.uint8(173): 182, np.uint8(118): 181, np.uint8(119): 181, np.uint8(124): 180, np.uint8(180): 180, np.uint8(144): 179, np.uint8(176): 179, np.uint8(179): 179, np.uint8(154): 177, np.uint8(152): 176, np.uint8(177): 175, np.uint8(183): 175, np.uint8(129): 174, np.uint8(165): 173, np.uint8(123): 172, np.uint8(174): 172, np.uint8(170): 170, np.uint8(181): 169, np.uint8(128): 167, np.uint8(138): 166, np.uint8(140): 166, np.uint8(166): 166, np.uint8(134): 165, np.uint8(184): 165, np.uint8(163): 165, np.uint8(153): 163, np.uint8(135): 162, np.uint8(142): 162, np.uint8(137): 162, np.uint8(175): 162, np.uint8(253): 161, np.uint8(127): 160, np.uint8(131): 160, np.uint8(151): 160, np.uint8(141): 158, np.uint8(160): 158, np.uint8(167): 158, np.uint8(171): 157, np.uint8(252): 156, np.uint8(143): 153, np.uint8(146): 152, np.uint8(157): 152, np.uint8(155): 152, np.uint8(161): 152, np.uint8(126): 150, np.uint8(133): 150, np.uint8(164): 150, np.uint8(156): 149, np.uint8(159): 148, np.uint8(178): 146, np.uint8(168): 146, np.uint8(169): 145, np.uint8(162): 143, np.uint8(149): 142, np.uint8(148): 142, np.uint8(150): 142, np.uint8(158): 141, np.uint8(136): 139, np.uint8(147): 138, np.uint8(139): 135, np.uint8(145): 134, np.uint8(130): 134, np.uint8(132): 133, np.uint8(254): 129})
```


EXPERIMENT-16

16. Shannon–Fano Coding and Arithmetic Coding

CODE:-

```
import cv2

import numpy as np

img = cv2.imread('input_image.jpg', 0)

values, counts = np.unique(img, return_counts=True)

prob = counts / counts.sum()

print("Symbol Probabilities:", prob)
```

```
python -u "c:\Users\mishr\image\ex1.py"
Symbol Probabilities: [2.63179012e-02 3.12075617e-02 3.38649691e-02 3.62970679e-02
4.07854938e-02 4.44351852e-02 4.65887346e-02 4.95270062e-02
5.11998457e-02 4.90007716e-02 4.76512346e-02 3.87075617e-02
3.35462963e-02 3.00362654e-02 2.72361111e-02 2.52523148e-02
2.34243827e-02 2.19020062e-02 2.07608025e-02 1.97716049e-02
1.87361111e-02 1.75748457e-02 1.68132716e-02 1.57854938e-02
1.48240741e-02 1.42330247e-02 1.35663580e-02 1.29081790e-02
1.18356481e-02 1.10324074e-02 1.03966049e-02 9.73611111e-03
9.11574074e-03 8.61651235e-03 7.97222222e-03 7.20216049e-03
6.27700617e-03 5.72145062e-03 5.00617284e-03 4.51157407e-03
3.96450617e-03 3.45447531e-03 2.99845679e-03 2.57716049e-03
2.35879630e-03 2.13117284e-03 1.96373457e-03 1.83641975e-03
1.74768519e-03 1.49382716e-03 1.46990741e-03 1.38040123e-03
1.25771605e-03 1.08873457e-03 1.01234568e-03 9.41358025e-04
9.34413580e-04 7.90895062e-04 6.93672840e-04 7.09876543e-04
5.69444444e-04 6.01851852e-04 5.27006173e-04 4.94598765e-04
4.84567901e-04 4.37500000e-04 4.46759259e-04 4.34413580e-04
3.85802469e-04 4.05092593e-04 3.73456790e-04 3.79629630e-04
3.63425926e-04 3.64197531e-04 3.51851852e-04 3.34104938e-04
3.27160494e-04 2.97067901e-04 3.15586420e-04 3.08641975e-04
2.73919753e-04 2.85493827e-04 2.85493827e-04 2.78549383e-04
2.55401235e-04 2.68518519e-04 2.68518519e-04 2.36111111e-04
2.59259259e-04 2.36111111e-04 2.66975309e-04 2.43055556e-04
2.36882716e-04 2.14506173e-04 2.21450617e-04 2.09104938e-04
2.02160494e-04 2.16820988e-04 2.02160494e-04 2.07561728e-04
2.02160494e-04 1.79783951e-04 1.62808642e-04 1.69753086e-04
1.80555556e-04 1.65895062e-04 1.67438272e-04 1.55864198e-04
1.80555556e-04 1.76697531e-04 1.64351852e-04 1.55092593e-04
1.65895062e-04 1.48919753e-04 1.56635802e-04 1.43518519e-04
1.63580247e-04 1.49691358e-04 1.39660494e-04 1.39660494e-04
1.42746914e-04 1.46604938e-04 1.45061728e-04 1.32716049e-04
1.38888889e-04 1.40432099e-04 1.15740741e-04 1.23456790e-04
1.28858025e-04 1.34259259e-04 1.03395062e-04 1.23456790e-04
1.02623457e-04 1.15740741e-04 1.27314815e-04 1.25000000e-04
1.07253086e-04 1.25000000e-04 1.28086420e-04 1.04166667e-04
1.28086420e-04 1.21913580e-04 1.25000000e-04 1.18055556e-04
1.38117284e-04 1.03395062e-04 1.17283951e-04 1.06481481e-04
1.09567901e-04 1.09567901e-04 1.09567901e-04 1.23456790e-04
1.35802469e-04 1.25771605e-04 1.36574074e-04 1.17283951e-04
```

```
python -u "c:\Users\mishr\image\ex1.py"
1.38117284e-04 1.03395062e-04 1.17283951e-04 1.06481481e-04
1.09567901e-04 1.09567901e-04 1.09567901e-04 1.23456790e-04
1.35802469e-04 1.25771605e-04 1.36574074e-04 1.17283951e-04
1.14969136e-04 1.17283951e-04 1.08796296e-04 1.14197531e-04
1.21913580e-04 1.17283951e-04 1.10339506e-04 1.27314815e-04
1.15740741e-04 1.33487654e-04 1.28086420e-04 1.21913580e-04
1.12654321e-04 1.11882716e-04 1.31172840e-04 1.21141975e-04
1.41203704e-04 1.40432099e-04 1.32716049e-04 1.25000000e-04
1.38117284e-04 1.35030864e-04 1.12654321e-04 1.38117284e-04
1.38888889e-04 1.30401235e-04 1.65895062e-04 1.35030864e-04
1.27314815e-04 1.47376543e-04 1.42746914e-04 1.46604938e-04
1.49691358e-04 1.55864198e-04 1.52006173e-04 1.49691358e-04
1.55864198e-04 1.54320988e-04 1.65123457e-04 1.42746914e-04
1.62808642e-04 1.52777778e-04 1.70524691e-04 1.72067901e-04
1.85956790e-04 1.85185185e-04 2.01388889e-04 1.87500000e-04
1.89814815e-04 1.86728395e-04 2.02160494e-04 2.29938272e-04
1.86728395e-04 2.11419753e-04 2.50771605e-04 2.19907407e-04
2.48456790e-04 2.70833333e-04 2.79320988e-04 2.56944444e-04
3.03240741e-04 3.00925926e-04 3.35648148e-04 3.20987654e-04
3.41820988e-04 3.54938272e-04 3.88888889e-04 3.92746914e-04
4.33641975e-04 4.45987654e-04 4.45216049e-04 4.09259259e-04
5.01543210e-04 5.30864198e-04 5.77932099e-04 5.39351852e-04
5.74845679e-04 4.80709877e-04 5.29320988e-04 5.11574074e-04
5.20833333e-04 5.22376543e-04 5.00000000e-04 4.83796296e-04
5.03086420e-04 4.42901235e-04 4.15895062e-04 4.16666667e-04
3.81944444e-04 3.80401235e-04 3.60339506e-04 3.17129630e-04
2.53858025e-04 2.36882716e-04 2.07561728e-04 1.68209877e-04
1.20370370e-04 1.24228395e-04 9.95370370e-05 2.83179012e-04]
PS C:\Users\mishr\image>
```